```
$ node breakfast.js --mode=async
```

JavaScript □同步

`// □□□□□□□□ Promise □ async/await`

sync
□隊等待

promise
未來值容器

await
同步的外表

event loop
幕後調度

```
$ cat menu.md
```

今日菜單

01 / [illegible]

[illegible] vs □同步開工

02 / Promise

三種狀態、then/catch、逾時保護

03 / async & await

[illegible] vs 並行

04 / Event Loop

VIP 通道與執行順序測驗

目標：看懂任何一段 `async` 程式碼的「實際執行順序」。

PART 01

□餐比喻

□□□□□□□□□□□□

同步：主執行緒被鎖死

```
1   function makeCoffee() {
2       console.log("開始煮咖啡...");
3       const start = Date.now();
4       while (Date.now() - start < 2000) {} // 阻塞!
5       console.log("咖啡好了! ");
6   }
7
8   function prepareBreakfastSync() {
9       makeCoffee(); // 2s
10      makeToast();  // 1.5s
11      fryEgg();     // 1s
12  }
```

CONSOLE — 4.5s

開始煮咖啡 . . .

咖啡好了!

開始烤吐司 . . .

吐司烤好了!

(這段期間頁面完全卡死)

□同步：全部同時開工

```
function makeCoffee() {
    return new Promise((resolve) => {
        console.log("開始煮咖啡...");
        setTimeout(() => {
            resolve("咖啡好了！");
        }, 2000); // 交給背景計時
    });
}

async function prepareBreakfastAsync() {
    const coffee = await makeCoffee();
    console.log(coffee);
    // ...吐司、蛋
}
```

```
CONSOLE — ~2s

開始煮咖啡...
開始烤吐司...
開始煎蛋...
咖啡好了！吐司烤好了！蛋煎好了！
```

主執行緒全程沒有被卡住

親眼比一次

```
$ node breakfast.js
```

選一種方式開始做早餐

□□ vs 非同步：早餐賽跑

▶ □步（一件做完才做下一件）

▶ 非□步（全部□時開工）

☕ 煮咖啡（2s）

🍞 烤吐司（1.5s）

🍳 煎蛋（1s）

同步總時間 = 2 + 1.5 + 1 = **4.5** □；非同步總時間 = max(2, 1.5, 1) = **2** □

PART 02

Promise

□□□□□□□□□□□□

三種狀態，單向變化

pending
待定：還在煮

fulfilled
已解決：resolve("□□")

rejected
已拒絕：reject("□□")

```
let myPromise = new Promise((resolve, reject) => {
    let success = true;
    setTimeout(() => {
        if (success) resolve("□□□");
        else reject("□□□");
    }, 2000);
});

myPromise.then(result => console.log(result))
         .catch(error => console.log(error));
```

逾時保護：Promise.race + AbortController

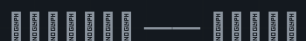
```
const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

const fetchWithTimeout = (url, { timeout = 3000 } = {}) => {
    const controller = new AbortController();
    const timer = delay(timeout).then(() => controller.abort());

    const request = fetch(url, { signal: controller.signal });

    return Promise.race([request, timer]).catch((error) => {
        if (error.name === "AbortError") return Promise.reject("請求逾時");
        return Promise.reject(error.message || "未知錯誤");
    });
};
```

Promise.race



AbortController

真的取消請求，不浪費資源

PART 03

async / await

□□□□□□□□□□□□

then → await

```
async function loadSpells() {
    try {
        const res = await fetch("https://hp-api.onrender.com/api/spells");
        const data = await res.json();
        console.log("spells", data.length);
    } catch (err) {
        console.error(err.message);
    }
}
```

兩者完全等價: await 只是 then 後續「攤平」同步樣子。長流 async/await，簡單鏈結 then。

串行：有相依才排隊

```
async function serial() {
    const coffee = await makeCoffee();
    const toast = await makeToast();
    return [coffee, toast];
}
```

TOTAL TIME

≈ 所有任務時□總和（2 + 1.5 = 3.5s）

□用
□□ token □□ API —— 後者依賴前者

並行：沒相依就同開

```
async function parallel() {
    const [coffee, toast] = await Promise.all([
        makeCoffee(),
        makeToast(),
    ]);
    return [coffee, toast];
}
```

TOTAL TIME

≈ 最慢□務（max(2, 1.5) = 2s）

□用
□□□□□□□□ API

Promise.all vs allSettled

	`Promise.all`	`Promise.allSettled`
一個失敗時	▯▯ reject▯全軍覆沒	照樣等所有完成
回傳	值的陣列	`{status, value/reason}` 陣列
適合	缺一不可的資料	「盡量收集」的儀表板

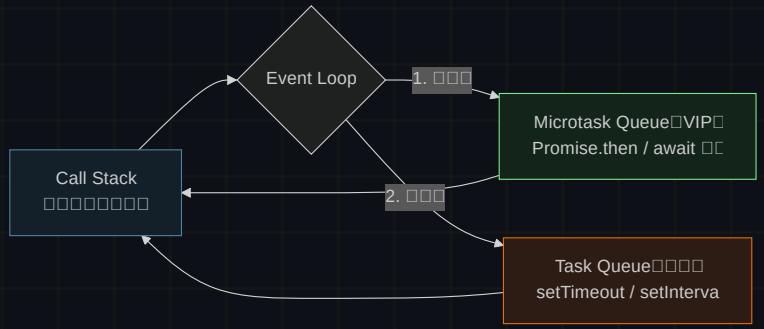
$ ▯▯▯▯▯`all` 是「全有或全無」，`allSettled` 是「盡力而為」。

PART 04

Event Loop

□□□□ VIP □□□□□□□□□□□

幕後調度：兩條佇列



Event Loop

1. □清空

Microtask Queue（VIP□
Promise.then / await 後□

Call Stack
同步代碼跑完清空

2. 才輪到

Task Queue（普通）
setTimeout / setInterva

微任務（VIP）

每回合先清空整條佇列

宏任務（普通）

微任務全空後，一次取一個

執行順序測驗

```javascript
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

ANSWER

1 → 4（同步代碼先跑完，Call Stack 清空）
→ 3（微任務 VIP 通道優先）
→ 2（宏任務最後，即使延遲是 0）

□□□□□□1 → 4 → 3 → 2

今日重點

非同步

耗時工作交給背景，主流程不阻塞

Promise

未來值容器：pending → fulfilled / rejected

□□ vs 並行

□□□□ await □□□□□□□ Promise.all

Event Loop

微任務全清空，才輪到宏任務

非同步的總時間 ＝ 最慢的那件事，不是所有事的總和。

Breakfast served.

□□□□fetch □ AbortController □□□□□□

```
$ node breakfast.js --mode=async
promise → await → race → event loop
```